



Modern Frontend with **React.js**

From Basics to Advanced



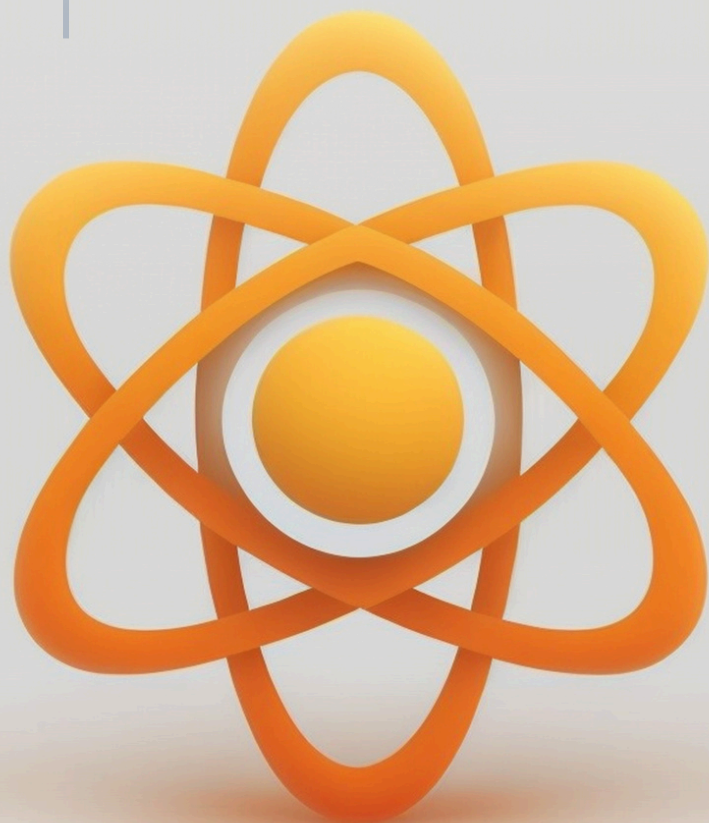
For software engineers &
DevOps professionals



Hands-on learning with
real-world projects



Industry-relevant
curriculum





What is React.js?

- React.js is a JavaScript library for building user interfaces, created by Facebook.
- It allows developers to create reusable UI components, enhancing efficiency in development.

Why use React.js?

- Component-based architecture 
- Virtual DOM for performance optimization 
- Large ecosystem & community support 

Did you know?

Over 40% of web applications use React.js as their primary frontend framework.



React Components & JSX



Understanding Components

- Components are reusable UI elements in React.
- There are two types of components:
 1. Functional Components
 2. Class Components



Example: Functional Component

```
function Welcome() {  
  return <h1>Hello, React Developers!</h1>;  
}
```



Practice Link: [React Component Challenges](#)



Props & State in React



Managing Data in React

- **Props:** Used to pass data from parent to child components.
- **State:** Manages component-specific data.



Example: Using Props

```
function Greeting(props) {  
  return <h1>Hello, {props.name}!</h1>;  
}
```



Example: Using State

```
import { useState } from 'react';  
function Counter() {  
  const [count, setCount] = useState(0);  
  return <button onClick={() => setCount(count + 1)}>Count: {count}</button>;  
}
```



React Hooks & Lifecycle Methods



React Hooks

- Hooks let you use state and lifecycle features in functional components.
- Common hooks include:
 1. `useState()` – Manages state within functional components.
 2. `useEffect()` – Handles side effects in components.



Example: Using `useEffect`

```
import { useEffect } from 'react';
function Example() {
  useEffect(() => {
    console.log('Component mounted');
  }, []);
  return <h1>React Hook Example</h1>;
}
```



React Router – Navigation in React Apps



Handling Navigation in React

- React Router allows users to navigate between pages in a React application.



Example: Using useEffect

```
import { BrowserRouter, Route, Routes } from 'react-router-dom';
function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
      </Routes>
    </BrowserRouter>
  );
}
```



Practice Link: [React Router Challenges](#)



State Management with Redux



Why use Redux?

- Centralized state management.
- Predictable state updates with reducers.
- Works well with large-scale applications.



Example: Simple Redux Store

```
import { createStore } from 'redux';
const reducer = (state = { count: 0 }, action) => {
  switch (action.type) {
    case 'INCREMENT': return { count: state.count + 1 };
    default: return state;
  }
};
const store = createStore(reducer);
```



Deployment & Performance Optimization



Optimizing and Deploying React Apps

Performance Optimization Tips:

- Code splitting with `React.lazy()`
- Using memoization with `React.memo()`
- Avoiding unnecessary re-renders with `useCallback()`



Deployment Options:

- Vercel (Best for React apps)
- Netlify (Fast & easy deployment)



Practice Link: [Deploy a React App](#)



Final Project – Build a Real-World App



Project: "React Task Manager App"

Features:

- Add, edit, and delete tasks.
- Persistent state with localStorage.
- Theming with styled-components.
- Hosted on Vercel.



Tech stack:

- Frontend: React.js + Tailwind CSS
- State Management: Redux
- Routing: React Router

Why Learn React.js with LearnBay?

- Learn from industry experts
- Hands-on projects & real-world scenarios
- Placement assistance & career mentorship

Start your React.js journey today!

Enroll now 

Join Learnbay's
**Full-Stack Software
Development Program!**

With **GenAI**'s hands-on experience



Learn how to integrate GenAI in your application



Job referrals in top companies with interview prep